CO523 Programming languages
Lec1

November 2025

B.A.K.Dissanayake

Department of computer Engineering

Faculty of Engineering

University of Peradeniya

Learning Outcomes

- After this lecture, students will be able to:
- Understand core principles behind programming languages
- Explain syntax, semantics, and language paradigms
- Compare static vs. dynamic typing
- Describe compilation vs. interpretation
- Understand scoping, binding, and memory management concepts

Concept

- What is programming language
- Why programming languages are needed
- How programming languages works

- Name 100 programming languages
  - **The top 20 essential programming languages**
  - **30 classic, up-and-coming, and niche programming languages**
  - **50 other programming languages worth mentioning**

⭐ What are Programming languages

- Programming languages are **formal, structured languages** used to communicate instructions to a computer. They allow humans to write programs that tell a computer **what to do and how to do it**.

⭐ Definition

- A **programming language** is a set of rules, symbols, syntax, and semantics that allows programmers to write instructions that a computer can execute.

⭐ Why do we need programming languages?

- Because computers only understand **binary (0s and 1s)**. Programming languages act as a **bridge** between human thinking and machine instructions.

⭐ Types of Programming Languages

1. Low-Level Languages

Closest to machine hardware.

- **Machine language** (binary code)
- **Assembly language**

Advantages: Fast, hardware control
Disadvantages: Difficult to write, error-prone

2. High-Level Languages

Easy to read and write, closer to human languages.
Examples:

- Python
- Java
- C++
- JavaScript
- C#

Advantages: Portable, readable, developer-friendly
Disadvantages: Slightly slower than low-level languages

3. Paradigm-Based Types

- **Imperative Languages**

Focus on *how* to do things.
Example: C, Python, Java

- **Object-Oriented Languages**

Use objects, classes, methods.
Example: Java, C++, Python

- **Functional Languages**

Focus on mathematical functions, no side effects.
Example: Haskell, Lisp, Scala

- **Logic Programming**

Based on rules and inference.
Example: Prolog

- **Scripting Languages**

Used for automation and rapid dev
Example: Python, JavaScript

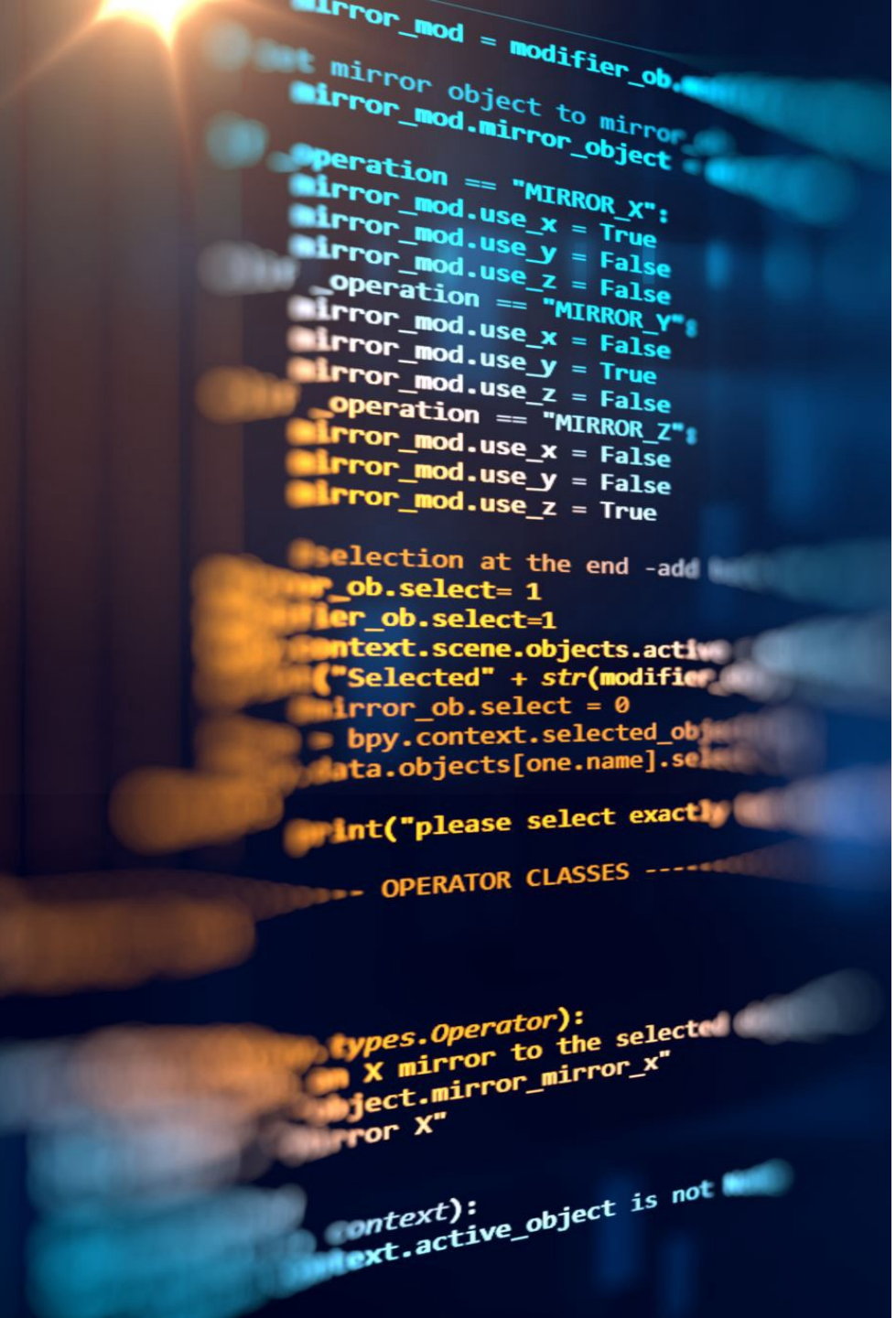
Name 100 programming languages

https://www.bairesdev.com/blog/top-programming-languages/

⭐ How Programming Languages Work

1. **Write code** in a programming language (e.g., Python).
2. **Translate** it to machine code.
   - Compiler (C, C++)
   - Interpreter (Python, Ruby)
   - Hybrid (Java, C# using Virtual Machines)
3. **Computer executes** the instructions.

https://youtu.be/RzjWJHwFL-E?si=zvV4mStfk2Oi2Urz

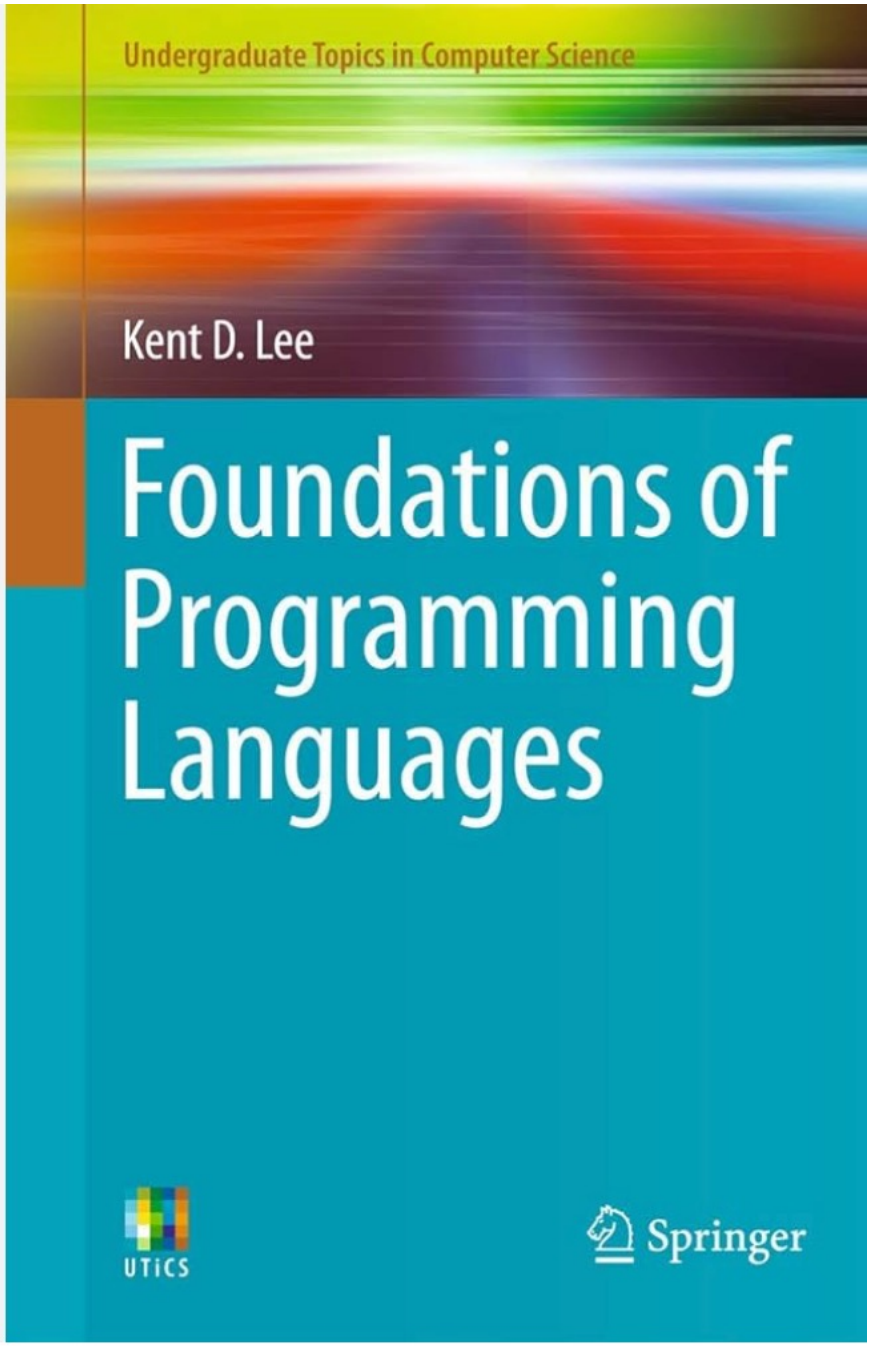


Concepts of Programming Languages

Concepts of Programming Languages

1. Explain the difference between syntax and semantics.
2. Compare static vs dynamic typing.
3. What is lexical scoping?
4. Describe advantages of functional programming.

PART 1: FOUNDATIONS OF PROGRAMMING LANGUAGES



Why Study Programming Languages?

Choose	Choose the right tool for the task
Understand	Understand trade-offs in language design
Write	Write better, safer, and efficient programs
Become	Become language-agnostic engineers

Language Design Goals

Language Design Goals

- Readability
- Writability
- Reliability
- Efficiency
- Portability
- Generality
- Maintainability

Language Evaluation Criteria

- Simplicity
- Orthogonality
- Data type support
- Syntax design
- Abstraction support
- Exception handling
- Security

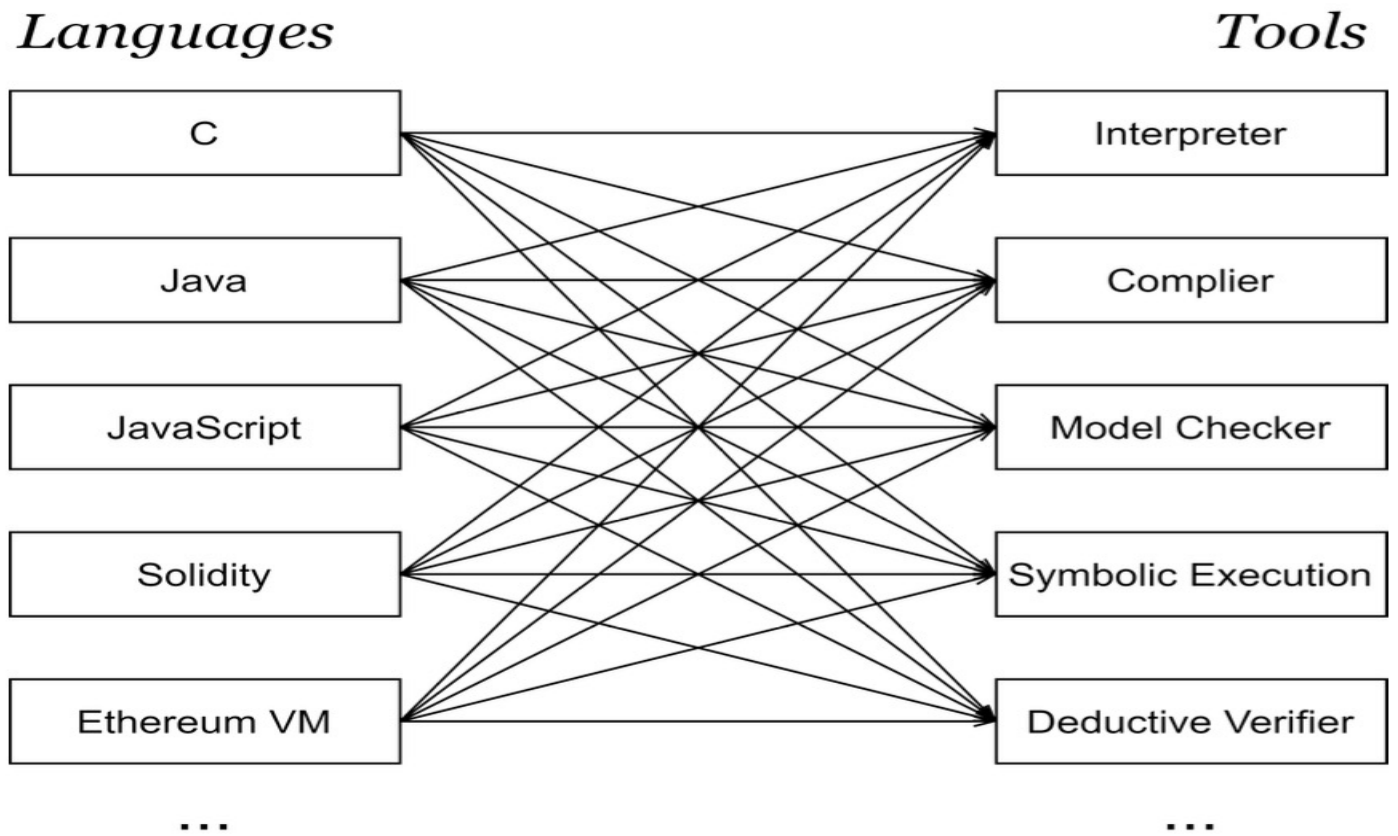


Fig. 1: The state-of-the-art of programming language design